Cilium 5주차
DNS · FQDN · L7 Policy 와 Envoy

L3/L4를 넘어서 — 언제, 왜 Envoy가 개입하는가

발표자 — 최원준

오늘의 목표와 4개의 질문

L3/L4를 넘어 **L7 Policy가 필요한 이유**와 **Envoy가 언제 개입하는지** 이해한다.

핵심 질문

1. DNS 기반 Policy는 어떻게 가능한가?
2. HTTP Policy는 어떻게 동작하는가?
3. eBPF만으로 L7이 가능한가?
4. Envoy는 언제 필요한가?

다룰 것

- L1~L7 페이로드가 변하는 방식
- DNS Proxy · FQDN · `toFQDNs`
- HTTP / gRPC L7 Policy
- Node-local Envoy
- eBPF와 Envoy의 역할 구분

4주차까지의 복습 — 우리가 아는 것

- **CNI**: Pod에 IP·veth 부여
- **eBPF datapath**: tc/XDP 훅에서 패킷 처리
- **kube-proxy 대체**: socket LB + Service map (O(1))
- **CiliumNetworkPolicy (L3/L4)**: identity(label) + port 기반, 커널에서 allow/drop

```
# 4주차: L3/L4 정책
egress:
- toEndpoints:
  - matchLabels:
      app: payment
  toPorts:
  - ports:
    - port: "8080"
      protocol: TCP
```

> 여기까지 정책이 보는 것은 **누가(identity) · 어디로(IP) · 어느 포트(L4)** 까지다.

그런데 이걸로 안 되는 두 가지

① 외부 도메인 통제

`api.github.com` 만 허용하고 싶다.
그런데 CIDR로 쓰려면?

```
- ipBlock:
    cidr: 140.82.121.3/32   # 내일 바뀜
```

CDN·anycast·LB 교체로 **IP가 수시로 변함** → 관리 불가

② 같은 포트, 다른 요청

`GET /public` 은 허용,
`POST /admin` 은 거부하고 싶다.

```
둘 다 목적지 :80/TCP
→ L3/L4로는 구별 불가
```

포트만 봐서는 **HTTP 메서드·경로**를 알 수 없다

> ①은 **DNS/FQDN Policy**, ②는 **HTTP L7 Policy** 로 푼다. 둘 다 패킷의 **더 안쪽(payload)** 을 봐야 한다.

기초 다지기 — 패킷은 L1~L7에서 어떻게 변하는가

정책이 "어디까지 볼 수 있나"는 곧 **패킷의 어느 껍질까지 열어보느냐**의 문제다.

```
   보내는 쪽 (캡슐화 = 껍질을 덧씌움)          받는 쪽 (역캡슐화)
L7 App     | HTTP: GET /pay  {JSON body}     |  ▲  애플리케이션이 최종 소비
L5/6 TLS   | [TLS 암호화] 세션·암호화          |  |  복호화·세션 종료
L4 TCP     | [TCP hdr | payload]  포트·SEQ    |  |  재조합(reassembly)·순서보장
L3 IP      | [IP hdr | ...]       출발/목적 IP|  |  라우팅
L2 Eth     | [MAC hdr | ... | FCS] 프레임     |  |  스위칭
L1 Wire    | 0101010101...  전기/광 신호       |  |  비트 수신
```

각 계층은 아래로 내려갈수록 **자기 헤더를 앞에 덧붙인다(encapsulation)**. 위로 올라오며 하나씩 벗긴다.

계층별로 "무엇이 보이는가"

계층	헤더/단위	그 계층에서 볼 수 있는 것	정책으로 쓰면
L3 IP	IP packet	출발/목적지 IP	CIDR 허용/차단
L4 TCP/UDP	segment	포트, TCP flags	포트 기반 정책
L7 HTTP	재조합된 스트림	method·path·header·host	경로/메서드 정책

- L3/L4는 **헤더의 앞부분 몇 바이트**만 보면 된다 → 커널(eBPF)이 패킷 단위로 즉시 판단 가능.
- L7은 **payload 안쪽 + 여러 패킷에 걸친 스트림**을 재조합·파싱해야 한다 → 훨씬 무겁다.

> 그래서 Cilium은 **L3/L4는 eBPF(커널), L7은 프록시(userspace)** 로 나눈다. 오늘의 전체 뼈대.

큰 그림 — 오늘 등장하는 3명의 처리자



- **eBPF** — 커널, L3/L4 전부
- **DNS Proxy** — agent 내장 Go, Envoy 아님
- **Envoy** — 노드 로컬, HTTP/gRPC L7
- 대부분 트래픽은 **eBPF만** 거친다

Part 1. DNS / FQDN Policy

이름으로 외부 접근을 통제한다

DNS란 — 계층적 이름 해석 시스템

이름(`api.github.com`) → IP(`140.82.121.3`) 를 매핑하는 전 세계 분산 데이터베이스. "인터넷의 전화번호부."

계층 구조 (root부터 위임)

```
. (root)
└─ com.              ← TLD
   └─ github.com.    ← authoritative
      └─ api         ← host
```

질의는 **root → TLD → authoritative** 순으로 위임되고, recursive resolver가 대신 순회한다.

주요 레코드 타입

타입	의미
`A` / `AAAA`	이름 → IPv4 / IPv6
`CNAME`	이름 → 다른 이름(별칭)
`NS`	도메인 담당 네임서버
`TXT`	임의 텍스트(SPF 등)

응답에는 **TTL**이 붙어 resolver가 그만큼 **캐시**한다.

FQDN이란 — 이름의 완전한 주소

FQDN = 루트부터 호스트까지 전부 명시한 **절대 이름**. 끝의 `.` 이 root(보통 생략).

```
api.github.com.
|   |      |  └ root ('.' 생략)
|   |      └─── TLD  (com)
|   └────────── SLD  (github)
└──────────────── host (api)
```

FQDN vs 상대 이름

- `api.github.com.` → **FQDN** (절대)
- `web`, `web.default` → **상대** 이름
- 상대 이름 + **search** 도메인 → FQDN

K8s 확장 (resolv.conf)

```
search default.svc.cluster.local …
options ndots:5
web → web.default.svc.cluster.local.
```

`ndots:5` : 점 < 5개면 search 먼저

DNS 복습 — 이름이 IP가 되는 과정

Pod의 `/etc/resolv.conf`:

```
nameserver 10.96.0.10   # kube-dns ClusterIP
search default.svc.cluster.local ...
options ndots:5
```

`curl api.github.com` 이면:

1. `getaddrinfo()` → DNS 질의(:53)
2. kube-dns가 IP 응답
3. 그 IP로 TCP 연결

관찰: 앱은 항상 **이름 → IP 변환을 먼저** 한다.

그렇다면 이 **DNS 질의를 가로채서**

- 어떤 이름을 허용할지 필터하고
- 응답으로 온 **IP를 기록**해두면?

→ 그 IP만 나중에 허용하면
도메인 단위 통제가 된다.

> Cilium FQDN Policy의 핵심 아이디어: DNS 응답을 엿봐서 "허용된 이름의 IP"를 실시간으로 알아낸다.

Cilium DNS Proxy — 정체는 Envoy가 아니다

```
// pkg/fqdn/dnsproxy/proxy.go
// DNSProxy is a L7 proxy for DNS traffic. ... A singleton is
// always running inside cilium-agent.
type DNSProxy struct { ... }
```

- **cilium-agent 프로세스 안의 Go 구현체** (miekg/dns 계열 라이브러리). Envoy와 무관.
- Pod의 DNS 질의(UDP/TCP 53)를 **eBPF가 TPROXY/`sk_assign`으로 프록시에 투명 리다이렉트**.
- 프록시는 질의를 **정책으로 필터** → 허용되면 kube-dns로 forward → **응답을 관찰**.

> **흔한 오해 정정**: "L7이니까 Envoy" ❌ — HTTP/gRPC만 Envoy. **DNS·Kafka는 agent 내장 Go 프록시**가 처리한다.

toFQDNs 정책은 항상 "2단 구성"

```yaml
egress:
# Part A — DNS 질의를 프록시로: "어떤 이름을 관찰/허용할지"
- toEndpoints:
  - matchLabels: {k8s:k8s-app: kube-dns, k8s:io.kubernetes.pod.namespace: kube-system}
  toPorts:
  - ports: [{port: "53", protocol: ANY}]
    rules:
      dns:
      - matchPattern: "*"
# Part B — 관찰로 알아낸 IP에 대한 L3 연결 허용
- toFQDNs:
  - matchName: "api.github.com"
  toPorts:
  - ports: [{port: "443", protocol: TCP}]
```

Part A 없이 Part B만 쓰면 동작하지 않는다. 프록시가 이름→IP를 못 봤으니 허용할 IP를 모른다.

이름이 허용 IP가 되기까지 — 동적 갱신



- 응답은 **datapath 갱신이 끝난 뒤** Pod에 전달(최대 `--tofqdns-proxy-response-max-delay`, 기본 100ms).
- 갱신 완료 전에 IP를 알고 그냥 연결하면(예: `curl --resolve`) **차단**된다.

matchName vs matchPattern

matchName — 완전 일치

```
toFQDNs:
- matchName: "api.github.com"
```

- `api.github.com` ✅
- `sub.api.github.com` ❌
- `github.com` ❌

matchPattern — 와일드카드

```
toFQDNs:
- matchPattern: "*.github.com"
```

- `api.github.com` ✅
- `gist.github.com` ✅
- `github.com` ❌ (`*`는 `.` 못 넘음)

- `*` 는 **하나의 라벨**만 대체하고 `.` 을 넘지 않는다. `github.com` 자체도 허용하려면 `matchName` 추가.
- Part A(`rules: dns`)와 Part B(`toFQDNs`)는 **같은 문법**을 쓰되 역할이 다르다: A=관찰 대상, B=연결 허용 대상.

FQDN — 운영 시 주의사항

함정	증상 / 해결
Part A 누락	toFQDNs가 조용히 안 됨 → DNS 규칙 먼저 확인
musl(alpine) 이미지	REFUSED 받으면 search 순회 중단 → `--tofqdns-dns-reject-response-code=nameError` (NXDOMAIN)
클러스터 내부 IP	`toFQDNs` 는 in-cluster IP 처리 불가 → `toEndpoints` / `toServices` 사용
TTL 너무 짧음	CDN이 TTL 30s → 잦은 갱신 → `--tofqdns-min-ttl` (기본 3600s)로 하한
agent 재시작	캐시 미보유 이름 일시 차단 → Standalone DNS Proxy(alpha)로 완화

- 캐시는 **노드 로컬**: Node A의 학습 결과는 Node A agent에만. 확인: `cilium fqdn cache list`.

Hubble로 DNS 흐름 보기

```
default/app:36657 -> kube-system/coredns:53
    L3-L4 REDIRECTED (UDP)            ← eBPF가 프록시로 우회
    to-proxy FORWARDED (UDP)
    dns-request FORWARDED (DNS Query api.github.com. A)     ← L7 이벤트
default/app:36657 <- kube-system/coredns:53
    dns-response FORWARDED (DNS Answer "140.82.121.3"
        TTL: 30 (Proxy api.github.com. A))                  ← 응답+IP
default/app:44754 -> api.github.com:443
    L3-Only FORWARDED (TCP Flags: SYN)  ← 학습된 IP로 연결 허용
```

```
hubble observe --protocol dns -f        # DNS 질의/응답 실시간
kubectl exec ds/cilium -- cilium fqdn cache list   # 학습된 이름→IP
```



Part 2. HTTP / gRPC L7 Policy

같은 포트, 다른 요청을 구별한다



HTTP Policy — toPorts에 rules.http 추가

```yaml
ingress:
- fromEndpoints:
  - matchLabels: {app: frontend}
  toPorts:
  - ports: [{port: "80", protocol: TCP}]
    rules:
      http:
      - method: "GET"
        path: "/public$"       # 나머지는 전부 403
```

필드	의미	생략 시
`method`	POSIX 정규식 (`GET` , `POST|PUT`)	모든 메서드
`path`	POSIX 정규식, `/` 로 시작	모든 경로
`host` / `headers`	Host 헤더 / 필수 헤더	무관

`path` 는 **부분 매칭**이다. `/public` 은 `/public-admin` 도 매칭 → 앵커 `^...$` 필수.

gRPC Policy = HTTP/2 위의 http 규칙

gRPC는 전송이 **HTTP/2**다. 각 RPC 호출은 이렇게 매핑된다:

```
POST  /<package>.<Service>/<Method>     Content-Type: application/grpc
```

따라서 **별도 `grpc:` 필드 없이 `http:`** 규칙으로 표현한다.

```yaml
toPorts:
- ports: [{port: "50051", protocol: TCP}]
  rules:
    http:                    # gRPC도 http 규칙
    - method: "POST"
      path: "/cloudcity.DoorManager/GetName"
    - method: "POST"
      path: "/cloudcity.DoorManager/RequestMaintenance"
    # SetAccessCode 호출 → gRPC status 7 PERMISSION_DENIED
```



L7 규칙은 allowlist — 위반은 drop이 아니라 거부 응답

allowlist 의미론

설정	동작
`rules: {}`	L7 우회 없음(L4만)
`http: [{}]`	전부 허용 + **가시성**
`http: [{GET /a}]`	그것만, 나머지 403

하나라도 규칙을 쓰면 **매칭 안 된 요청은 거부**.

거부 = 프로토콜 응답

- HTTP → `403 Forbidden`
- gRPC → `PERMISSION_DENIED(7)`

패킷 드롭(타임아웃)과 달리
의도된 거부임이 명확 → 디버깅 유리

> 함정: 같은 포트에 **L4-only 규칙**이 따로 있으면 그게 더 넓게 허용해 L7 규칙이 무력화될 수 있다 → 규칙을 합쳐라.

HTTPS(암호화)는? — TLS Visibility

Envoy는 **평문**을 파싱해야 한다. 외부로 나가는 HTTPS는 기본적으로 내용을 못 본다.

```
egress:
- toFQDNs: [{matchName: "httpbin.org"}]
  toPorts:
  - ports: [{port: "443", protocol: TCP}]
    terminatingTLS:    # Pod <-> Envoy 구간 TLS 종료 (Pod는 내부 CA 신뢰 필요)
      secret: {namespace: kube-system, name: httpbin-tls-data}
    originatingTLS:    # Envoy <-> httpbin.org 구간 TLS 재개시
      secret: {namespace: kube-system, name: tls-orig-data}
    rules:
      http: [{}]       # 이제 평문으로 HTTP 규칙 적용 가능
```

- Cilium 1.17+ : 인증서는 **SDS**로 참조 전달(중복 제거). 이전엔 inline(NPDS).

Part 3. Node-local Envoy

L7을 실제로 처리하는 프록시



Envoy란 — 프로그래밍 가능한 L4/L7 프록시

- Lyft가 만든 고성능 프록시(현재 CNCF 졸업, C++). Istio·Cilium·Gateway 구현체가 공유하는 **데이터 플레인**.
- **동적 설정(xDS)**: 재시작 없이 리스너·라우트·클러스터를 API로 주입 → 제어 플레인이 실시간 조종.
- 요청이 지나는 경로:

```
Listener(수신 포트) → Filter Chain(TLS·HTTP 파서)
   → Route(host/path 매칭) → Cluster(목적지 그룹) → Endpoint(실제 Pod IP)
```

> Cilium은 사용자가 Envoy 설정을 직접 안 쓴다. **CiliumNetworkPolicy의 L7 규칙을 agent가 xDS로 번역해 Envoy에 주입한다.**



핵심 차이 — Pod당이 아니라 노드당 1개

```
[Istio Sidecar]   Pod마다 Envoy      → Pod 20개면 Envoy 20개
[Cilium]          노드당 Envoy 1개   → Pod 20개여도 Envoy 1개
```

모델	Envoy 위치	개수
Istio Sidecar	Pod마다	Pod 수
Cilium	노드당	노드 수
Istio Ambient	ns/서비스	워크로드군

- 노드 1개 Envoy 20개 → **1개**
- L7 규칙 **없는** Pod는 Envoy를 **아예 안 거침** (eBPF fast path)
- 8주차 Ambient ztunnel/waypoint와 대조

배포 형태 — embedded → 독립 DaemonSet

예전 (~1.15): embedded

```
cilium-agent Pod
├── cilium-agent (Go)
└── envoy (서브프로세스)
```

agent 재시작 → Envoy도 재시작 → **L7 연결 끊김**

지금 (1.16+ 기본): 독립 DS

```
노드
├── cilium-agent Pod
└── cilium-envoy Pod
```

agent 재시작해도 **Envoy 생존** → L7 연결 유지

- 통신은 두 모드 모두 **UNIX 도메인 소켓** (네트워크 오버헤드 없음).
- 분리 이점: 독립 lifecycle·리소스 격리·보안 격리. `--set envoy.enabled=true` (1.14+ opt-in, 1.16+ 기본).

eBPF는 어떻게 Envoy로 넘기는가

```
Pod A ──→ tc eBPF hook (veth)
          │  이 flow에 L7 규칙 있나? (policy map 조회)
          ├─ 없음 → eBPF fast path로 직행 (Envoy 안 거침)
          └─ 있음 → skb mark = MARK_MAGIC_TO_PROXY, redirect
                    │
                    ▼  TPROXY (원래 목적지 보존) / sk_assign
               cilium-envoy (노드 로컬 리스너 :proxyPort)
                    │  BPF Metadata filter로 source Identity 전달
                    │  HTTP 파싱 → method/path/header 정책 평가
                    ├─ allow → 원래 목적지 Pod로 upstream
                    └─ deny  → 403 / PERMISSION_DENIED
                    ▼
                 Pod B
```

Envoy를 직접 태우는 길 — CiliumEnvoyConfig

- **자동 경로**: `CiliumNetworkPolicy`의 L7 규칙 → agent가 xDS로 번역 (사용자는 Envoy 몰라도 됨).
- **고급 경로**: `CiliumEnvoyConfig` / `CiliumClusterwideEnvoyConfig` CRD로 **Envoy 리소스를 직접** 정의.

```
kind: CiliumEnvoyConfig
spec:
  services: [{name: my-svc, namespace: default}]
  resources:
  - "@type": type.googleapis.com/envoy.config.listener.v3.Listener
    # ... 라우팅·레이트리밋·서킷브레이커 등
```

> **Ingress·Gateway API도 이 노드 Envoy 위에 구현된다** (7주차 예고). L7 라우팅·TLS 종료가 전부 같은 Envoy.

Part 4. eBPF vs Envoy

왜 이렇게 역할을 나눴는가



eBPF만으로 HTTP를 못 하는 이유

eBPF의 실행 모델

- 패킷 **1개마다** 독립 호출 (per-packet)
- 호출 간 상태 공유 없음
- verifier 제약: **bounded loop**만, **스택 512B**, 명령어 상한

HTTP/2·gRPC의 현실

- 요청이 **여러 TCP 세그먼트**에 걸침 → **재조합** 필요
- HTTP/2 **HPACK**: 연결 수명 내내 유지되는 동적 테이블 필요
- 스트림 **멀티플렉싱**

```
세그먼트1: "GET /api/v1/us"   세그먼트2: "ers HTTP/1.1\r\nHost: ex"   세그먼트3: "ample..."
   → eBPF는 세그먼트1만 보고 판단해야 하는데, 요청이 아직 안 끝남
```

> 커널에서 TCP 스트림 재조합 + HTTP/2 상태기계를 구현하는 것은 verifier 제약상 비현실적.

Envoy는 왜 가능한가 — 그리고 DNS는 왜 예외인가

Envoy (userspace)

- 표준 TCP **소켓** 사용
- 커널이 **재조합한 바이트 스트림**을 `read()`
- 메모리·스택 제약 없음 → HTTP/2·HPACK 상태기계 완비

DNS는 왜 eBPF+Go로 OK?

- 단순 **요청-응답**, 고정 포트, 짧은 payload
- 스트림 재조합 부담 없음
- agent 내 **Go 프록시**면 충분 (Envoy 불필요)

> 결론(질문 3): **eBPF만으로는 HTTP/gRPC L7 불가.** eBPF는 리다이렉트 판단까지, 파싱은 userspace(Envoy). DNS는 단순해서 agent 내장 Go 프록시로 해결.

역할 구분표 — 누가 무엇을 처리하나

기능	처리자	위치
L3/L4 포워딩·정책, Service LB, NAT, 암호화	eBPF	커널
대부분의 관측(flow, Hubble L3/L4)	eBPF	커널
HTTP/gRPC method·path·header 정책	Envoy	userspace / 노드당
Ingress·Gateway API L7 라우팅, TLS 종료	Envoy	userspace / 노드당
DNS proxy · FQDN, Kafka(레거시)	cilium-agent 내장 Go	userspace / 노드당

> 한 문장: **커널에서 될 건 eBPF로, 프로토콜 파싱이 필요할 때만 프록시로.** L7 중에서도 DNS는 agent Go, HTTP/gRPC는 Envoy.

"Envoy는 언제 필요한가" — 실제 빈도

```
① Pod → Pod (L3/L4만)            → eBPF fast path          (Envoy X)
② Pod → Service (L4 LB)            → eBPF socket LB          (Envoy X)
③ FQDN egress 정책                 → DNS proxy(agent Go)     (Envoy X)
④ HTTP/gRPC method·path 정책       → TPROXY → Envoy          (Envoy O)
⑤ Ingress / Gateway API            → TPROXY → Envoy          (Envoy O)
```

- ①②③이 클러스터 트래픽의 **대부분**. Envoy가 실제로 관여하는 건 ④⑤ — **명시적으로 L7을 쓸 때만**.
- 그래서 사이드카처럼 "모든 트래픽 × 프록시" 곱셈 비용이 없다.

> 답(질문 4): **HTTP/gRPC L7 규칙, 또는 Ingress/Gateway를 쓸 때만 Envoy가 개입한다.**

정리 — 4개의 질문에 답한다

질문	답
DNS 기반 Policy는?	agent 내장 **DNS Proxy**(Go)가 질의를 엿봐 이름→IP를 학습, `toFQDNs`가 그 IP를 허용 (2단 정책)
HTTP Policy는?	`toPorts.rules.http` (method/path/header) → **Envoy**가 파싱·집행, 위반은 403
eBPF만으로 L7?	**아니오.** per-packet·verifier·재조합 한계. eBPF는 리다이렉트까지, 파싱은 userspace
Envoy는 언제?	**HTTP/gRPC L7 규칙·Ingress/Gateway** 일 때만. 노드당 1개, 나머진 eBPF fast path

> **커널에서 될 건 eBPF, 프로토콜 파싱이 필요할 때만 Envoy. DNS만 예외적으로 agent 내장 Go 프록시.**

다음 주 & 우리 환경에 적용한다면

6주차 예고 — Hubble

- 오늘 본 dns-request / http-request L7 이벤트를 **흐름 단위로 관측**
- "왜 막혔나 / 어느 계층에서 끊겼나"

생각해볼 것

- 외부 API 연동에 `toFQDNs` 적용 지점?
- HTTP L7까지 갈 트래픽이 실제로 얼마나?
- HTTPS 비중 → TLS visibility 필요성?

참고: Cilium Docs — DNS-Based policies / Layer 7 / Envoy · cilium/cilium `pkg/fqdn` · v1.16 Release Notes